SmartBIS 운영 수집 설계 (초안, GPS 없음) rev 11

> 상태: **초안**. rev 9 변경 사항은 부록에 적었다(2026-10-27). 호출 규칙(4장)은 Codex 수정 요청의 기준이다. 실제 TAGO 호출 검증은 아직 하지 않았다. 확인 사항은 10장.

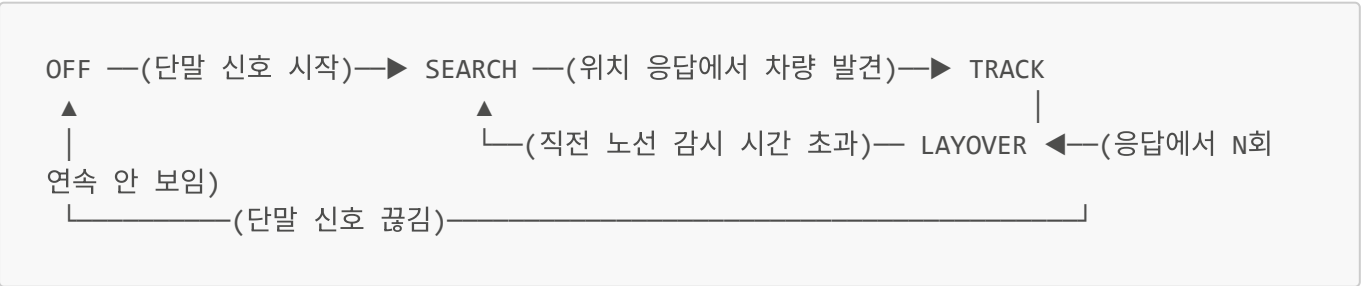
1. 전제

- 차량 단말에 GPS가 없다 → 차량 위치는 TAGO 위치 API(`getRouteAcctoBusLcList`)로만 알 수 있다
- 한도: API별 하루 10,000건(개발계정), 운영 예산 9,500건
- 운행 시간: 관내 06:15~21:00, 100번 05:30~. 관내 64개 노선(하루 편도 333회)과 100번(장성↔광주, 평일 84회·휴일 74회)
- 관내 노선의 48%(31개)는 하루 3회 이하, 89%(57개)는 10회 이하 운행한다
- 관내는 차량 18대가 **순번별 고정 운행표**(순번 1~10, ①~⑧)에 따라 여러 노선을 이어서 운행한다(관내총괄 시트의 숫자를 순번으로 해석함. 10장 1번 확인 필요)
- 정류장 간격은 평균 약 1.2분이다(관내 정류장 통과 하루 약 10,000회, 추정)
- TAGO 노선 ID는 **105개**로 확인했다(12장). 관내 88개(노선번호 48개)와 100번 계열 17개(이름 8개)다. 운수사 노선과 TAGO ID를 정류장 순서로 맞춘 결과, 관내 운행의 69%는 TAGO ID 1개로 확정되고 31%는 후보가 2~7개다(13장)
- 위치 API는 **노선 단위**로 조회한다: 1회 호출로 그 노선의 모든 차량(차량번호·위치)이 온다
- 도착정보 API 응답에는 차량번호가 없다 → 같은 노선·같은 방향 차량이 여러 대면 구분하기 어렵다
- TAGO 데이터 갱신 주기는 10~20초

2. 관리하는 것 / 관리하지 않는 것

대상	방식
차량–노선 배정	**관리하지 않는다.** 순번 운행표를 한 번 등록해 두고, 차량이 어느 순번인지는 운행 중 발견된 (노선, 시각)으로 자동 매칭한다
순번 운행표·노선 시간표	운수사 자료(관내총괄, 관내, 100번 평일/휴일)를 등록한다. 개편 때만 갱신한다
운행 준비 중인 차량	단말 신호(heartbeat)로 자동 판단한다 (단말 전원이 시동과 연동될 때)

3. 차량 상태



상태	뜻	호출
OFF	단말이 꺼져 있음	없음

상태	뜻	호출
SEARCH	단말은 켜졌는데 어느 노선인지 모름	스윕 대상
TRACK	노선을 알고 있음 (순번을 알면 이후 운행도 예측)	운행 중인 노선을 주기적으로 조회
LAYOVER	운행을 마치고 대기 중	순번을 알면 다음 출발 2분 전까지 호출 없음. 모르면 직전 노선만 감시(최대 30분)

- **정류장 관측 누락 보간 (3장·5장 공통):** BIS 데이터는 LTE 끊김이나 위치 이탈로 빠질 수 있으므로 완전하다고 가정하지 않는다. 같은 `route_id` 안에서 연속된 두 관측의 `node_order`가 증가하고, 사이 누락 정류장이 **5개 이하이며 30분 이하**이면 사이 정류장을 통과로 보고 추정 시각을 넣는다(`interpolated=true`). 누락이 6개 이상이거나 30분을 넘으면 '구간 미확인'으로 둔다. 순번이 감소하거나 `route_id`가 바뀌면 보간하지 않는다. 보간은 상태 추적과 구간 소요 시간 학습에만 쓰며, 노선 운행 여부 판정(일치·불일치·미관측)에는 실제 관측만 쓴다.

4. 위치 API 호출 (①위치 수집 프로세스만 수행, rev 9)

- **적용 범위:** ①위치 수집 프로세스만 TAGO 위치 API를 호출한다. 앱 API(⑤)와 화면(⑥)은 TAGO를 호출하지 않고 DB만 읽는다.
- **호출 시간대:** 05:15~21:15에만 호출한다. 21:15~05:15에는 위치 호출을 하지 않으며, 호출 시도가 생기면 로그만 남긴다.
- **예산 가드:** 일 9,500건(운영 예산). 재시도 최대 2회, 간격 10초, 실패한 시도도 예산에 포함한다. 사용량은 `guard_usage_daily`에 기록하고 재시작 시 당일 값을 복원한다. 상한에 도달하면 그날 위치 호출을 멈추고 로그를 남긴다.
- **lane 분리:** 세 lane은 각자 스레드와 간격을 가진다. 도착정보 호출과 throttle을 공유하지 않는다.
  - **Lane A (장성100 상시):** 장성100 ID 목록(10장 8번)의 ID를 05:15~21:15 동안 10분 주기로 조회한다.
  - **Lane B (시간표 창):** 관내 88개 ID와 시간표가 확인된 변형 100번 ID를 출발 시각 창 안에서만 10분 주기로 조회한다. 창 밖에서는 호출하지 않는다.
  - **Lane C (백업 스윕):** 105개 ID 전체를 06:00~21:00 동안 30분마다 1회전한다. 1회전은 ID를 2초 간격으로 순차 호출하며(약 3.5분), 끝나면 다음 주기까지 쉰다. 간격은 설정값이다.
- **429 대응:** 429 응답이 나오면 lane C의 간격을 2초 → 3초 → 5초로 단계적으로 올리고 로그를 남긴다. 단계 값은 설정값이다.
- **창 규칙 (lane B):**
  - 기준: 기점 또는 종점의 출발 시각(운수사 시간표, rev 9에서 확인).
  - 창 시작 = 출발 시각 − 15분(관내), − 10분(변형 100번).
  - 창 종료 = 출발 시각 + 편도 소요 시간 + 10분.
  - 편도 소요 시간은 노선별 시간대 중앙값을 학습해 쓴다. 학습 전에는 정류장 수 × 1.2분으로 추정한다.
  - 같은 ID의 창이 겹치면 하나로 합친다.
- **시간표 없는 ID:** 이름이나 시간표를 확정하지 못한 ID는 lane C만 적용한다. 장성100 ID는 lane B에 넣지 않는다.
- **패턴·배차 판정:** 스윕과 창의 관측 결과는 ③이 전날 밤에 해석한다. ①은 판정 결과인 `route_call_plan`만 읽으며 판정 로직을 알지 못한다.
- **관측 모드:** 8장을 따른다. 105개 10초 순환은 폐기한다.

5. 도착정보 API

- 정류장 간격이 짧아(평균 약 1.2분) 정류장마다 조회하면 한도를 넘는다(관내+100번 하루 약 13,000~14,000건, 추정)

- 조회하는 경우: ① 운행을 시작할 때 ② 다음 정류장까지 예상 소요가 3분 이상일 때 ③ 예정 시각이 지났는데 아직 통과하지 않았을 때
- **방문당 상한 (rev 8 추가):** 정류장 한 번 방문에 도착정보 조회는 최대 2회다. 이 상한은 정책 코드와 DB CHECK(0~2, V019)로 고정하며 설정으로 바꿀 수 없다.
- 그 밖의 짧은 구간은 쌓인 위치 기록에서 학습한 **정류장 간 소요 시간**(시간대별 중앙값)으로 계산한다. 학습 전에는 마지막 도착 예정 시각에서 경과 시간을 빼서 보여준다
- 응답에서 `arrprevstationcnt`(남은 정류장 수)가 차량 위치와 맞는 값을 고른다. 관내 노선은 같은 노선·방향에 우리 차량이 동시에 1대인 경우가 대부분이라 혼동이 적다
- 정류장 관측 누락은 3장 보간 기준(연속 5개 이하, 30분 이하)으로 메운 뒤 구간 소요 시간을 학습한다. 보간 행은 학습에만 쓰고 운행 판정에는 쓰지 않는다. 수집률(관측 정류장 수 ÷ 노선 정류장 수)은 summary 지표로만 쓰며 노선 제외 기준이 아니다.

6. 예산 추정 (rev 9, 추정)

관내 창 건수(2,058)는 rev 8 값을 임시로 쓴다. 출발 시각 기준으로 재계산해야 한다. 장성100 상시 대상 ID 수 N은 10장 8번에서 확정한다.

항목	위치 API (건/일)	비고
Lane A 장성100 상시 (10분, 05:15~21:15, ID당 96회)	96 × N	N=2면 192, N=17이면 1,632
Lane B 관내 창 (10분)	약 2,058	rev 8 값, 재계산 필요
Lane B 변형 100번 창	약 300	출발 약 30회 × 창당 약 10회 (추정)
Lane C 백업 스윕 (30분, 06:00~21:00, 30회전 × 105)	3,150	2초 간격
합계, N=2	**약 5,700**	여유 약 3,800
합계, N=17 (최악)	**약 7,100**	여유 약 2,400

- 재시도 여유는 500 이상을 유지한다. 예산이 부족하면 lane C 주기를 먼저 늘린다(20분이면 45회전, 4,725건. N=17 기준 합계 약 8,700).
- rev 8 관측 모드(105개 10초 순환, 약 8,640건 추정)와 달리 야간과 창 밖 호출이 0이다.
- 이전 도착정보 추정(4,000~5,000건)은 관측으로 다시 확인하기 전까지 쓰지 않는다.
- 변형 100번 시간표 수치는 10-22 광주 패턴 파일 기준이며, 평일·휴일 모두 하루 1~5회다.
- 100번 정류장 수와 도착정보 조회 비율은 추정이다. 관측으로 확인한다.

7. 한도를 넘을 때의 선택지 (효과가 큰 순)

1. **운영계정 전환 + 트래픽 증가 신청:** 공공데이터포털에 활용사례를 등록하고 신청한다. 얼마나 늘려 주는지는 신청해 봐야 안다
2. **지자체 BIS 센터·운수사 위치 데이터를 직접 연계:** 버스의 BIS 단말이나 디지털 운행기록계에는 GPS가 있고, TAGO도 이 데이터를 모아서 제공한다. 직접 받으면 한도 문제가 사라진다
3. **외장 GPS 수신기(USB/블루투스):** 단말이 외부 장치 연결(USB OTG 등)을 지원하는지 확인이 필요하다
4. **정밀도 낮추기:** 2정류장마다 확인하고, 중간은 쌓인 기록에서 얻은 구간 소요 시간으로 추정한다. 호출은 약 절반이 되지만 정확도가 떨어진다
5. **도착정보로 통과 판정:** 같은 노선·방향에 동시에 1대만 다닐 때만 쓸 수 있다. 운수사 자료상 관내 노선 대부분이 이 조건에 해당한다(하루 10회 이하 노선 89%). 위치 API 사용은 줄지만 도착정보 사용은 늘어난다

8. 관측 모드 (먼저 할 일)

- **기간:** 평일 3일 + 토요일·일요일
- **설정:** 4장의 lane A·B·C 규칙을 그대로 쓴다. 105개 10초 순환 설정은 쓰지 않는다. 관측과 운영의 호출 규칙이 같아야 관측 결과를 운영에 쓸 수 있다.
- **저장:** 우리 차량만이 아니라 응답에 나온 **모든 차량**(차량번호, 노선, 정류장 순번, 시각)
- **얻는 값:**
  - 순번 운행표와 실제 운행의 차이(지연·결행·순번 교체)
  - 대응표에서 후보가 여러 개인 16개 노선(관내 운행 103회)이 실제로 어느 TAGO ID로 잡히는지, 시간표에 대응되지 않는 TAGO ID 6개에 차량이 나타나는지(13장)
  - 창 밖 운행: lane C가 잡은 차량 중 창 밖에서 발견된 건수(창 규칙 검증)
  - 출발 시각과 실제 첫 관측 시각의 차이(창 시작점 검증)
  - 정류장 간 실제 소요 시간(시간대별)
  - 실제 운행 시간의 합 R
  - 같은 노선·방향을 동시에 달리는 비율
- **확인 필요:** 지금 수집기가 응답의 모든 차량을 저장하는지(Codex에게 보고 요청). 현재 코드가 10초 순환이므로 이 규칙에 맞춘 수정이 먼저다.

9. 기존 코드와의 관계

- `TagoQuotaGuard`: 예산 가드로 유지한다. lane A·B·C마다 간격 설정(`window`, `min-gap-ms`)을 따로 둔다. 남은 예산에 따른 자동 간격 조정은 lane C에만 적용한다.
- `ArrivalService`의 2분 기준: 운영 방식에서는 위치가 2분 넘게 갱신되지 않을 수 있다 → "TRACK 상태이고 도착 예정 시각이 유효함" 기준으로 바꿔야 한다
- 다음 정류소 찾기: TAGO 순번은 노선 ID 안에서 0부터 끝까지 이어지고, 상·하행이 바뀌는 지점에서도 다시 시작하지 않는다(12장). 같은 방향 조건을 빼고 `route_id`와 `node_order + 1`로 찾아야 회차 지점의 404가 사라진다
- 방향 값: 31개 ID는 `updowncd=2`(방향 구분 없음)다. 방향이 UP/DOWN인 행만 쓰는 지금 조회 방식에서는 이 노선들(100번 계열 12개 ID 포함)이 항상 404가 될 수 있다
- 순번 0: 노선 정류소 순번이 0부터 시작한다. 수집기는 순번 1 미만 위치를 버리므로(308줄), 위치 API 순번도 0부터라면 기점에 있는 차량이 빠진다. 위치 API 응답으로 확인이 필요하다
- 테스트 단계(단말 3대, 10초 간격)와 운영 방식의 차이는 설정값과 상태 판단으로만 둔다. 테스트 전용 분기 코드는 만들지 않는다
- `TagoApiRequestThrottle`: 위치·도착·스윕이 한 throttle을 공유하면 스윕이 장성100·창 호출을 뒤로 민다. lane별 throttle로 나눈다.

10. 결정·확인 필요

확인됨 (rev 9)

- 광주 패턴 기준 파일은 `10-22 광주 운행 패턴.csv`다.
- 광주 패턴의 시각은 기점 또는 종점의 출발 시각이다.

미해결

1. 순번(1~10, ①~⑧)과 차량번호의 관계: 고정인지, 매일 바뀌는지. 바뀐다면 그날 배차를 기록한 자료가 있는지(관내 창 정확도에 영향)
2. ①~⑧ 표시의 의미(영업소·차종 등)와 18대가 관내 전체 차량인지. 100번 운행 차량 수와 단말 설치 대상 여부
3. 관내 토·일·공휴일 운행표가 따로 있는지, 이 자료가 현재 기준인지(6번 노선에 '2022.02.04부터 운행' 메모가 있음). 휴일 관내 창은 이 자료 없이 정할 수 없다

4. 단말 전원이 시동과 연동되는지, 운영할 단말 대수
5. 운영계정 신청 가능 여부, 지자체 BIS 센터·운수사와 위치 데이터 연계 협의 가능 여부
6. 관측 모드를 진행할 시점(규칙은 4장에서 확정)
7. 87번(사창–생촌, 하루 1회)은 TAGO에 없다. 지금도 운행하는지, 단말 설치 대상인지
8. **장성100 ID 목록(Lane A 대상):** 100번 계열 17개 ID 중 장성100 이름의 ID. 102·103이 장성100으로 추정된다(평일 43+42회, 운수사 평일 84회와 1회 차이). 확정 전까지 장성100 ID는 lane C만 적용한다
9. **변형 100번 이름 미확인 ID:** 104·106·108~112·154 등 이름이 확인되지 않은 ID. 이름 확정 전까지 lane C만 적용한다. 107·118·119·153은 이름이 확인되었다(노선도_TA 또는 TAGO 자료)
10. 운수사 자료 정리: 시간표의 `91-1`이 `관내 총괄 편집` 시트에서는 `91-2`로 되어 있다(같은 노선인지). 편집 시트의 12번 행은 경유 정류장이 비어 있다

11. 운수사 자료 분석 (`노선(사이텍).xlsx`)

시트	내용
관내	관내 64개 노선: 기점·종점, 거리, 편도 횟수, 기점/종점 출발 시각, 경유 정류장 순서
관내총괄	군민운수 관내 총괄 운행표: 방면별로 순번(1~10, ①~⑧)과 출발 시각
100번 평일 / 휴일	장성↔광주 100번, 23.6km, 평일 84회·휴일 74회, 05:30부터
터미널	장성공용버스터미널 주소·전화

- 순번 18개는 각각 하루 14~21회 출발하며 여러 방면을 이어서 운행한다. 첫 출발 06:15~07:20, 마지막 출발 17:50~21:00
- 예: 순번 ④는 10:00·12:00·14:00·16:00에 같은 방면으로 출발해 50분 뒤 돌아오는 2시간 주기를 반복한다
- 자료 정리 시 주의: 70·71·93번은 편도 횟수가 시각 개수의 절반으로 적혀 있다(왕복 기준으로 보임). 시트 합계 315회와 시각 기준 333회가 다르다. 출발 시각 옆 괄호(`(청룡)`, `(물메)` 등)는 경유 변형이다
- 관내총괄은 셀 병합과 방면 블록이 섞여 있어 자동 추출 결과 일부에 중복·누락이 있을 수 있다. 등록 전에 순번별 운행표를 운수사와 대조한다

12. TAGO 노선 자료 분석 (`TAGO_노선도.xlsx`, 2026-10-02 수집)

- 두 시트(노선별 경유정류소 5,339행 / 중복값 제거 105행)를 비교했다. 노선 ID 105개가 양쪽에 모두 있고, 정류장 순서와 정류장 수가 모두 일치한다. 정류장 순서가 같은 ID는 없다 → **105개는 중복이 아니라 서로 다른 노선 ID다**

구분	노선 이름	노선 ID
관내	48개 (1~95번)	88개
100번 계열	8개 (녹진·마령·비아·삼서·월곡·월정·장성·진원100)	17개
합계	56개	**105개**

구조	ID 수	특징
왕복형	74개	ID 하나에 기점→회차→기점이 모두 들어 있다. `updowncd` 0(상행)→1(하행)으로 한 번 바뀌고, 순번은 0부터 끝까지 이어진다

구조	ID 수	특징
편도형	31개	`updowncd=2`(방향 구분 없음). 가는 방향과 오는 방향이 서로 다른 ID다(예: 20번 `...029` 터미널→사거리, `...030` 사거리→터미널). 종점에 도착하면 차량이 반대 방향 ID로 옮겨 간다

- 운수사 대응: 운수사 관내 64개 = 기본 번호 48개 + 세부 번호 16개(10-1 등). TAGO 이름 48개는 기본 번호와 모두 일치한다. 세부 번호와 TAGO 변형 ID는 개수가 꼭 맞지 않는다(예: 10번은 TAGO 7개 / 운수사 4개, 90번은 TAGO 4개 / 운수사 3개) → 이름이 아니라 **정류장 순서로 대응**해야 한다
- 운수사 87번은 TAGO에 없다
- 정류장 951개, ID당 정류장 10~113개(평균 51개), 좌표 누락 없음
- 수집 설계 반영: 스윕 대상은 105개 ID다. 편도형 노선에서 운행이 끝나면 반대 방향 ID를 먼저 확인한다(3장 LAYOVER의 "직전 노선"에 반대 방향 ID를 포함)

13. 운수사 노선 ↔ TAGO 노선 ID 대응 (`TAGO_노선도.xlsx`의 `관내 총괄 편집` 시트)

- 방법: 운수사 노선별 경유 정류장 순서(63개 노선)를 같은 번호의 TAGO ID 정류장 순서와 비교했다. 정류장 이름은 약칭이 섞여 있어(예: 북일면 ↔ 북일면사무소) 비슷한 이름도 같은 정류장으로 보고, 순서가 일치하는 비율로 점수를 매겼다. 편도형은 가는·오는 ID를 이어 붙여 비교했다

결과	운수사 노선	관내 하루 운행
TAGO ID 1개로 확정	47개 (12번은 편집 시트에 정류장이 없지만 TAGO ID가 1개뿐이라 대응)	229회 (69%)
후보 여러 개	16개: 10·10-2·10-3(후보 7개), 90(4개), 11·11-1·11-2·13·13-1·90-2(3개), 32·70·71·81·82·91(2개)	103회 (31%)
TAGO 없음	87번	1회

- 후보가 여러 개인 이유: 같은 번호의 TAGO ID끼리 정류장 순서가 94~99% 같다(예: 10번 7개 ID). 정류장 순서만으로는 구분할 수 없다 → 관측으로 "어느 시각의 운행이 어느 ID로 잡히는지" 확인한다
- 시간표에 대응되지 않는 관내 ID 6개: 2번 `...003`, 4번 `...006`, 61번 `...064`, 63번 `...067`, 70번 `...070`, 71번 `...073`. 지금은 운행하지 않는 옛 변형이거나 경유 변형(`(청룡)` 등)일 수 있다. 스윕 우선순위를 가장 낮게 두고, 관측에서 차량이 보이면 올린다
- 시간대별 운행 예정 ID(후보 전부 포함, 추정): 관내 88개 중 06시 36개, 07시 59개(최대), 낮 시간 42~54개, 19시 33개, 20시 13개, 21시 6개. 스윕은 지금 시간대의 운행 예정 ID부터 한다
- 이 대응은 자동 비교 결과다. 등록 전에 운수사와 대조한다

부록. rev 8 변경 사항 (2026-10-26)

반영한 내용

- 4장: DB 적용 방식 추가. backend는 Flyway 미사용, `server/database/migrations`를 V001→V019 순서로 `psql` 적용. 빈 DB 시뮬레이션 통과(테이블 28개, 운영과 동일 목록).
- 5장: 정류장 방문당 도착 조회 최대 2회 상한 추가. 정책 코드와 DB CHECK(0~2, V019)로 고정.

변경하지 않은 내용

- 1~3장, 6~9장의 수치와 설계 판단은 rev 7 그대로다.
- 10장 결정·확인 1~9는 모두 미해결이다. 이번 시뮬레이션으로 해결된 항목은 없다.
- 6장 예산(8,679건)은 추정치이며 실제 TAGO 호출로 검증하지 않았다.

시뮬레이션 범위와 제외 항목

- 확인: 빈 DB 마이그레이션, backend 기동(`Started SmartBisApplication`, `APPLICATION FAILED` 없음).
- 제외: HMDM, nginx, Android, 실제 TAGO 호출. 시뮬레이션 설정은 TAGO 호출을 모두 끈 상태였다.

부록. rev 9 변경 사항 (2026-10-27)

반영한 내용

- 4장: 105개 10초 순환 관측 모드 폐기. 호출 규칙을 lane A(장성100 상시), lane B(시간표 창), lane C(백업 스윕, 2초 간격, 30분 주기)로 나눔. 위치 호출 시간대 05:15~21:15.
- 4장: 창 규칙 확정(기점·종점 출발 시각 기준, 시작 = 출발 − 15분(관내)/− 10분(변형), 종료 = 출발 + 편도 소요 + 10분).
- 6장: 예산 재계산(N=2 약 5,700, N=17 약 7,100, 추정).
- 8장: 관측 모드를 운영 호출 규칙과 같게 함.
- 9장: throttle을 lane별로 분리.
- 10장: 광주 패턴 기준 파일과 시각 기준을 확인됨으로 표시. 장성100 ID 목록, 변형 100번 이름 미확인 ID를 미해결로 추가.

변경하지 않은 내용

- 1·2·3·5·7장. 3장 상태 전이는 rev 8과 같다.
- 6장의 관내 창 건수 2,058은 rev 8 값을 임시로 쓴다.

현재 상태

- 운영 소스는 아직 105개 10초 순환(관측 모드)이다. 이 규칙에 맞는 코드 수정 전까지 야간 호출을 포함한 낭비가 계속된다.
- 실제 TAGO 호출 검증은 하지 않았다.

13. 설정 분리: 배포 설정 / 기능 스위치 / 운영 정책 / 노선 자료 (rev 10, 2026-10-27)

13.1 원칙

- **배포 설정 (`.env`, `compose.yaml`)**: 서버마다 다른 값과 비밀값만 둔다. DB 접속, 포트, 인증서 경로, 로그 경로, 타임존, `TAGO_SERVICE_KEY`.
- **기능 스위치 (비상 차단용)**: `TAGO_ENABLED`, `TAGO_VEHICLE_LOCATION_ENABLED`, `TAGO_ARRIVAL_ENABLED`. 기본값은 안전 쪽(`false`)이다. 스위치가 `false`이면 정책 값과 무관하게 해당 기능은 호출하지 않는다.
- **운영 정책 (정책 테이블, 애플리케이션 관리)**: 호출 시간 창, lane별 주기, 스윕 주기, 429 백오프 단계, 창 앞뒤 분, 재시도 횟수와 간격, 관측 모드, 03:00 카탈로그 동기화 주기. 재기동 없이 변경한다.
- **노선 자료 (시간표 테이블)**: 노선별 출발 시각, 편도 소요 시간 학습값, lane A 대상 ID(장성100), lane B 대상 ID(관내·확정 변형 100번), 노선 구조(왕복형·편도형). 운수사 자료 등록으로만 갱신한다.
- **코드 고정 (정책 아님)**: 도착정보 방문당 2회 상한(V019 CHECK), 야간 위치 호출 금지 구간(21:15~05:15), 순번 0 기준, 상태 전이 규칙, 일일 예산 하드 상한 검사 로직.

13.2 우선순위와 조회 방식

- 우선순위: **노선 값 > 서비스 값 > 프로필 기본값 > 코드 안전 기본값**.
- 정책 테이블 `tago_policy`: `policy_key`, `value`, `value_type`, `scope_type`(GLOBAL / SERVICE / ROUTE), `scope_id`, `profile_name`, `updated_by`, `updated_at`, `note`.
- 변경 이력 `tago_policy_history`를 둔다. 변경마다 이전 값, 새 값, 변경자, 시각을 남긴다.
- 반영: 60초마다 재로딩하거나 관리 API 호출 시 즉시 반영한다. 재기동은 필요 없다.
- 조회 실패 시: 마지막으로 읽은 정상 값을 쓴다. 정상 값이 없으면 **위치 수집을 끈 상태**(안전 기본값)로 둔다.
- 정책 값이 코드 고정 규칙을 어기면(예: 도착 방문당 3회) 저장을 거부하고 로그를 남긴다.

13.3 정책 키 (초기값은 현재 설계 값)

키	초기값	설명
`location.laneA.enabled`	true	장성100 상시 수집
`location.laneA.interval_min`	10	lane A 주기(분)
`location.laneA.window`	05:15~21:15	lane A 호출 시간대
`location.laneB.lead_min.regular`	15	관내 창 시작 = 출발 − 15분
`location.laneB.lead_min.variant`	10	변형 100번 창 시작 = 출발 − 10분
`location.laneB.tail_min`	10	창 종료 = 출발 + 편도 소요 + 10분
`location.laneB.interval_min`	10	lane B 주기(분)
`location.laneC.enabled`	true	백업 스윕
`location.laneC.window`	06:00~21:00	스윕 시작 가능 시간대
`location.laneC.cycle_min`	30	스윕 주기(분). 예산 부족 시 먼저 늘린다
`location.laneC.gap_sec`	2	ID 간 간격(초). 1~3초 범위
`location.laneC.backoff_429_sec`	2,3,5	429 시 단계 간격
`budget.daily_limit`	9500	일일 위치 호출 상한(계정 제약, 변경은 승인 후)
`budget.retry_max`	2	재시도 횟수
`budget.retry_gap_sec`	10	재시도 간격
`observation.mode`	false	관측 모드(105개 순환). 운영에서는 false 고정
`catalog.sync_cron`	0 0 3 * * *	카탈로그 동기화 주기
`arrival.enabled`	true	도착정보 사용

코드 고정(정책 키 아님): 위치 호출 금지 구간 21:15~05:15, 도착 방문당 상한 2회, 순번 0 기준.

13.4 프로필

- 프로필 4종: 대도시형, 중소도시형, 도농복합형, 농촌형. 프로필은 정책 키의 기본값 묶음이다.
- 프로필 이름만으로 값을 정하지 않는다. 기준 데이터는 노선 수, 노선별 하루 운행 횟수, 출발 시각 분포다.
- 장성군의 프로필은 노선 자료로 확정한다(도농복합형 후보). 확정 전까지는 현재 설계 값(13.3 초기값)을 쓴다.

- 서비스 구분: **차량 서비스**(위치 수집, 창 계산), **정류소 서비스**(도착정보), **공공시설 서비스**(노선과 무관한 조회). 정책 키는 서비스 단위로 덮어쓸 수 있다.
- 프로필 적용: 배포 시 프로필 하나를 고르고, 필요한 키만 정책 테이블에서 덮어쓴다.

13.5 단계

- **단계 0 (오늘 밤, 승인 후 `.env`/`compose.yaml`만 변경)**: 비상 차단. 위치 수집과 관측 모드를 끈다. 03:00 동기화 차단은 Codex가 소스에서 방법을 확인한 뒤 정한다(추측 값 사용 금지).
- **단계 1 (DB 작업 완료 후, 아침 프로세스 기동 전, 소스 수정)**: 13.2~13.3의 정책 조회 로직을 추가하고, 기존 `.env` 정책 값을 코드 기본값과 정책 테이블 초기값으로 옮긴다. 호출 규칙(4장)은 정책 값을 읽도록 바꾼다. 테스트는 프로필별 값, 우선순위, 조회 실패 시 안전 기본값, 시각 고정 호출 규칙을 검증한다. 실제 TAGO 호출은 하지 않는다.
- **단계 2 (낮 운영 안정화 후)**: 관리 API와 화면, 변경 이력 조회, 프로필 선택.
- **결정 필요**: 정책 테이블(V020) 적용을 오늘 밤 DB 작업에 포함할지. 기본안은 **포함하지 않고** 단계 1에서 별도 적용하며, 적용 전 백업을 다시 받는다.
- 단계 1의 빌드·배포(새 이미지 기동)는 별도 승인 대상이다.

13.6 오늘 밤 `.env` 변경 항목 (승인 대상, 값은 실제 현재값을 확인 후 정함)

- `TAGO_LOCATION_OBSERVATION_MODE=false`
- `TAGO_VEHICLE_LOCATION_ENABLED=false` (21:00 이후 낮 수집은 이미 끝난 상태. 아침 재개는 단계 1 배포 후)
- `TAGO_CATALOG_SYNC_CRON`: 차단 방식 확인 전까지 변경하지 않는다
- `TAGO_ENABLED`, `TAGO_ARRIVAL_ENABLED`: 변경하지 않는다(도착 기능 유지)

부록. rev 10 변경 사항 (2026-10-27)

- 13장 추가: 설정 4분류, 정책 테이블과 우선순위, 정책 키 초기값, 프로필과 서비스 구분, 단계 0~2, 오늘 밤 `.env` 항목.
- 4장 호출 규칙의 수치는 정책 키로 옮긴다(단계 1). 수치 자체는 변경하지 않았다.

14. 0번 기준 검증과 테스트 기준 (rev 11, 2026-10-27)

> 이 장은 **설계서 기준**이다. 이번 Codex 로직 수정 요청 범위가 아니며, 검증 실행은 로직 구현 뒤 별도 승인으로 진행한다.

14.1 노선 유형별 기점·종점 기대값

- **왕복형(74개 ID)**: 한 ID 안에서 기점(UP 0번)과 종점(DOWN 마지막 순번)이 **같은 정류소 node_id**다. 예: TSB342000143 UP 757=0, DOWN 757=76. TSB342000001 기점 757=0, 회차 579=11, 종점 757=22. 같은 이름이라도 node_id가 다르면 맞은편 정류소다.
- **편도형(31개 ID)**: `updowncd=2`. 한 ID는 한 방향이고 순번은 0부터 끝까지 이어진다. 기점과 종점은 서로 다른 정류소다. 방향 반대 ID는 별도 ID다.
- **공통**: 순번은 노선 ID 안에서 0부터 끝까지 이어진다. 상·하행이 바뀌는 지점에서도 다시 시작하지 않는다(12장).

14.2 연속성 판정 규칙 (노선 단위)

- `min(node_order) = 0`
- `count(*) = count(DISTINCT node_order)` (중복 없음)

- `max(node_order) - min(node_order) + 1 = count(*)` (빠짐 없음)
- 세 조건을 모두 만족하면 통과. 샘플 노선(`SAMPLE_ROUTE_001`)은 검사 대상이 아니다.
- 방향 경계에서 정류소 이름이 바뀌는 것은 정상이다. 연속성 판정은 순번만 본다.

14.3 기점 이름 대조 규칙

- 기준: 엑셀 `TAGO_노선도.xlsx`의 노선별 기점·종점 이름과 DB의 기점·종점 `node_id`.
- 조인: `bus_route_stops.node_id` = `bus_stops.node_id`(기본키). 이름은 `bus_stops.node_name`.
- 판정: 0번 이름과 node_id가 엑셀 기점과 같으면 통과. `node_name`이 NULL이면 `bus_stops`에 없는 정류소로 **미확인**으로 둔다.
- 대조 대상: 노선 105개 전체를 원칙으로 하되, 1차는 왕복형 대표 ID(0001, 0143)와 편도형 대표 ID(0002, 0044, 0105, 0111)로 시작한다.

14.4 0번 기준 도착정보 확인

- 대상: 0번 정류소 중 하루 운행이 확인된 노선 2개 이내.
- 방법: 도착정보 API를 1~2회만 호출한다(방문당 상한 2회, 5장). `arrprevstationcnt`가 차량 위치와 맞는지 본다.
- 실제 TAGO 호출은 **운영 승인 후 낮 시간**에만 한다. 개발 테스트에서는 호출하지 않는다.

14.5 테스트 목록과 통과 기준

번호	테스트	통과 기준
T1	시각 고정: 창 안·밖 호출	창 안 호출 1건, 창 밖 0건
T2	21:15~05:15 호출 금지	해당 시각 위치 호출 0건, 로그만 남김
T3	창 겹침 병합	같은 ID의 겹치는 창이 하나로 합쳐짐
T4	lane C 1회전	105건, ID 간격 2초
T5	429 백오프	2→3→5초 순서로 간격 상향, 로그 기록
T6	창 계산 예시	118 출발 07:15 → 시작 07:05(변형 −10분). 153 출발 06:45 → 시작 06:35
T7	장성100 제외	102·103이 lane B 대상에 없음
T8	0번 기준 입력	0143 UP 757 순번 0, DOWN 757 순번 76을 입력으로 한 단위 테스트(DB 없음)
T9	빌드·테스트 실행	exit 코드, Tests run / Failures / Errors 실제 숫자. 실패 0

- 테스트 실행 기록은 로그 파일로 저장하고, 보고에는 tail 발췌와 숫자를 쓴다.

15. 작업 범위 분리 (rev 11)

15.1 이번 Codex 요청 범위 (로직 수정만)

- 4장 수집 로직: lane A·B·C 시간표 창 방식, 21:15~05:15 호출 금지, lane별 throttle 분리, 429 백오프
- 0번 기준 잔여 하한 1 → 0 (`normalizeStopOrders`, `firstAvailableOrder`, `positionFrom`, `TagoVehicleLocationService` nodeOrder 검사)
- 위 로직에 필요한 단위 테스트(T1~T8)
- 제외: 소스 외 설정 변경, DB 접속, 컨테이너 재기동, 실제 TAGO 호출, push

15.2 이번 범위 밖 (설계서에만 기록, 하나씩 확인 후 진행)

- 13장 정책 조회 로직(`tago_policy`, V020, 우선순위, 60초 재로딩, 조회 실패 시 위치 수집 끄기)
- 14장 검증 실행(노선별 기점 대조 전수, 도착정보 0번 확인)
- 위치 수집 재개(`TAGO_VEHICLE_LOCATION_ENABLED=true`): 4장 로직과 T1~T9가 통과하고 승인된 뒤에만
- 테스트 서버 이전 준비

15.3 확정되지 않은 입력 (로직 구현 시 기본값을 쓰고 설정으로 둔다)

- 장성100 대상 ID 목록(N): 확정 전까지 lane C만 적용
- 이름 미확인 ID(104·106·108~112·154): lane C만 적용
- 편도 소요 시간: 학습값이 없으면 정류장 수 × 1.2분
- 시각 기준: 기점 또는 종점 출발 시각(운수사 자료 확인됨)
- 관내 창 건수 2,058은 임시값(재계산 필요)

15.4 체크리스트

- 이번 요청 범위 확정(로직 수정만): ☑
- 0번 잔여 하한 수정 및 T8 통과: ☐
- lane A·B·C 창 방식 수정 및 T1~T7 통과: ☐
- 테스트 실행 숫자(exit, Tests run, 실패 0): ☐
- 정책 조회(13장) 별도 확인: ☐
- 검증 실행(14장) 별도 승인: ☐
- 위치 수집 재개 승인: ☐

부록. rev 11 변경 사항 (2026-10-27)

- 14장 추가: 기점 기대값, 연속성 판정, 이름 대조 규칙, 도착정보 확인, 테스트 T1~T9와 통과 기준.
- 15장 추가: 이번 Codex 요청 범위(로직 수정만)와 범위 밖 항목, 확정되지 않은 입력, 체크리스트.
- 1~13장 설계 값은 변경하지 않았다.